UNIVERSIDAD MAYOR DE SAN ANDRÉS

CARRERA DE INFORMÁTICA

BASE DE DATOS II

INF- 244

Grupo 4: Recolección de Basura

Estudiantes: **Imanol Marcelo Moya Condori**

Emit Anel Sharom Viracka Alanoca

Keiko Andrea Ticona Condori

Joel Pérez Quelali

Dario Daniel Condori Vargas

Ailza Nadiezhda Huarachi Ibañez

Docente: **Lic. ZEBALLOS ABASTO JOSE LUIS**

LA PAZ - BOLIVIA

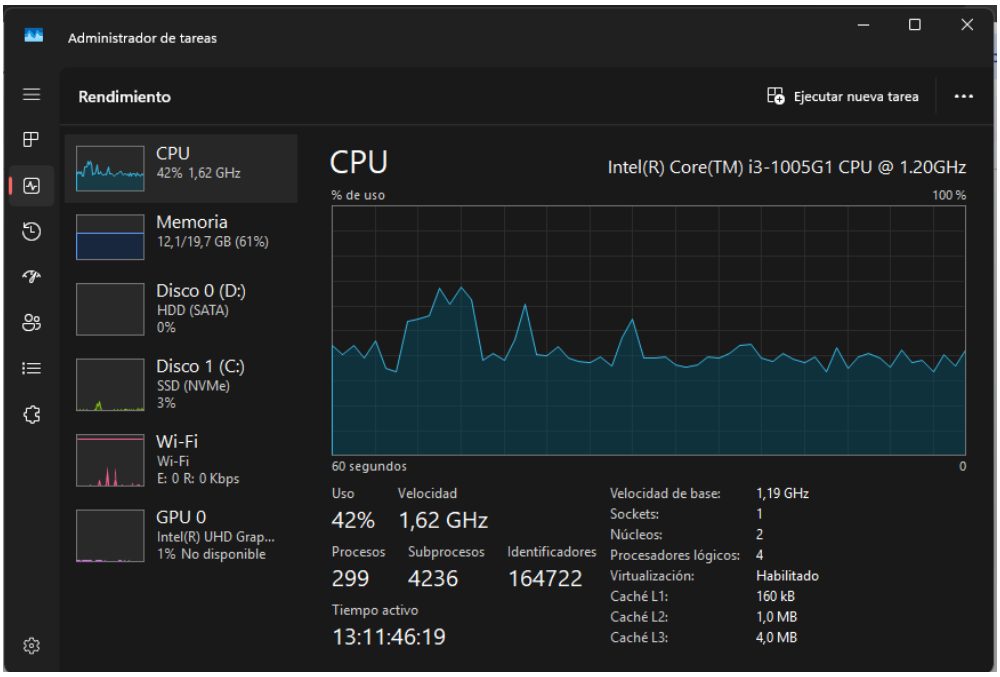
2026

GUÍA DE INSTALACIÓN Y DESPLIEGUE OFFLINE

1. Requisitos Previos en la Máquina Destino

Antes de conectar la memoria USB, la computadora del usuario debe cumplir con:

1. **Sistema Operativo:** Windows 10 u 11 (64 bits).
2. **Virtualización Habilitada:** Verificar en Administrador de Tareas →pestaña Rendimiento → CPU →Virtualización: Habilitado.



3. **Software Base Instalado:**
   - **Docker Desktop** (debe estar abierto y con el ícono de la ballena en verde abajo a la derecha)

```
Descargando: Subsistema de Windows para Linux 3.0.3
Instalando: Subsistema de Windows para Linux 3.0.3
Se ha instalado Subsistema de Windows para Linux 3.0.3.
Instalando componente opcional de Windows: VirtualMachinePlatform

Herramienta Administración y mantenimiento de imágenes de implementación
Versión: 10.0.26100.9549

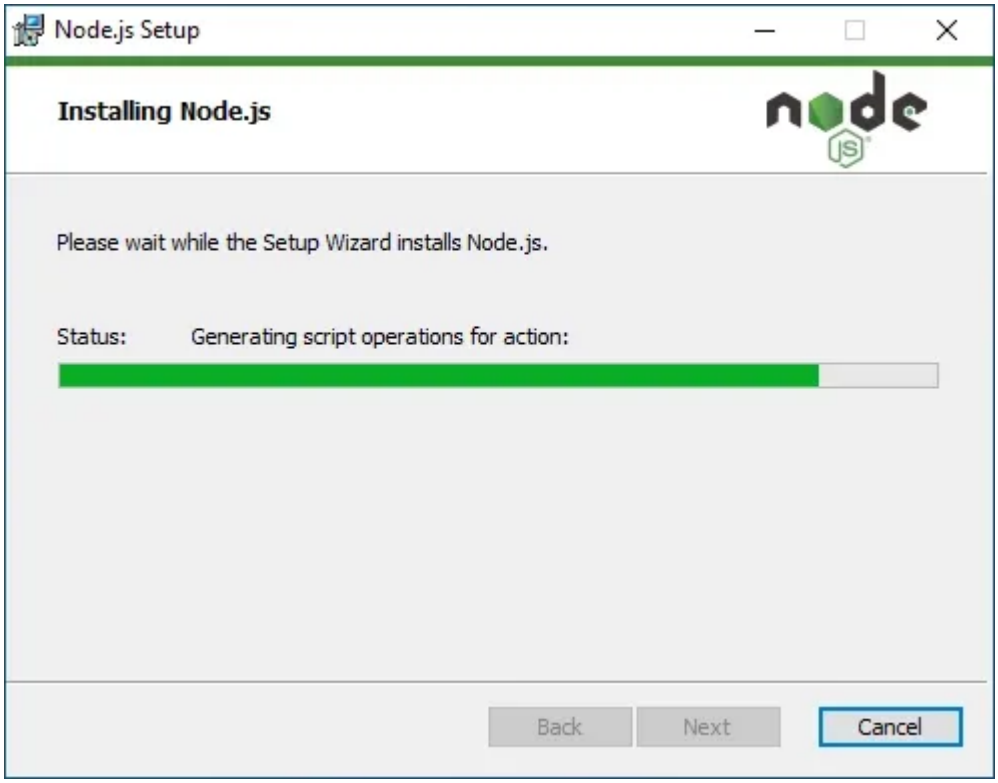
Versión de imagen: 10.0.26100.9711

Habilitando características
[==========================100.0%==========================]
La operación se completó correctamente.
La operación solicitada se realizó correctamente. Los cambios se aplicarán una vez que se reinicie el sistema.
La operación solicitada se realizó correctamente. Los cambios se aplicarán una vez que se reinicie el sistema.
PS C:\Users\Emit> wsl -l
Subsistema de Windows para Linux no tiene distribuciones instaladas.
Para resolverlo, instale una distribución con las instrucciones siguientes:

Use 'wsl.exe --list --online' para enumerar las distribuciones disponibles
y "wsl.exe --install <Distro>" para instalar.
PS C:\Users\Emit> wsl --list
Subsistema de Windows para Linux no tiene distribuciones instaladas.
Para resolverlo, instale una distribución con las instrucciones siguientes:

Use 'wsl.exe --list --online' para enumerar las distribuciones disponibles
y "wsl.exe --install <Distro>" para instalar.
PS C:\Users\Emit>
```

- **Node.js** (versión 18, 20 o superior instalada para ejecutar los scripts de Backend y Frontend).



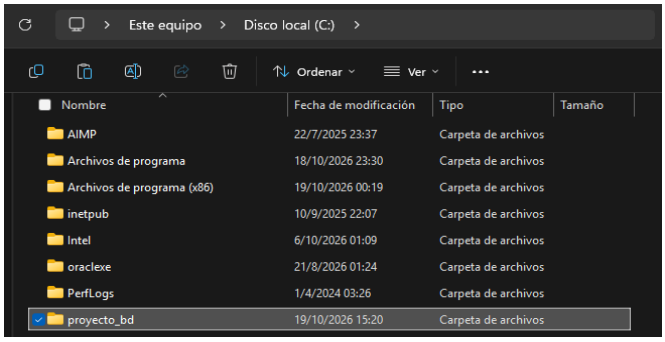
2. Paso a Paso: Despliegue en 3 Pasos desde el USB

Paso 1: Copiar la carpeta a la raíz del disco C:\

Conectar la memoria USB.

1. Copiar la carpeta completa proyecto_bd que viene en el flash y pegarla directamente en la raíz de su disco local C:\.
2. La ruta final **debe ser obligatoriamente**:

   C:\proyecto_bd



(No colocarla en el Escritorio ni dentro de otras subcarpetas para evitar problemas con rutas relativas).

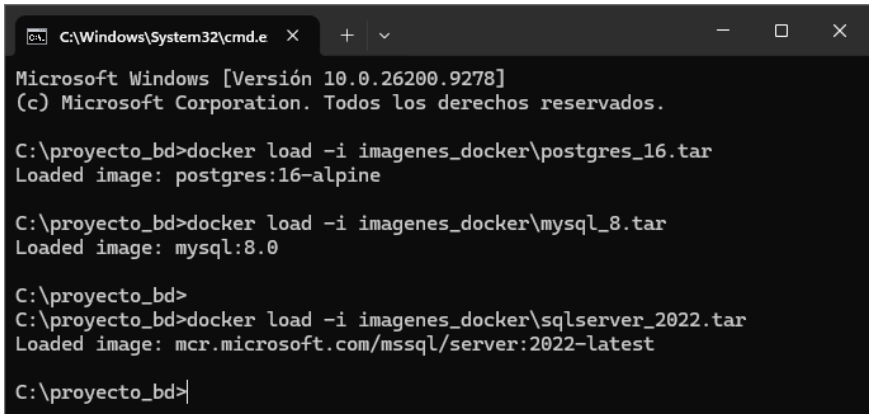
Paso 2: Cargar las Imágenes de Docker (100% Offline)

Como el aula o laboratorio puede no suelen contar con una red estable de internet, las imágenes ya vienen descargadas dentro de la carpeta imagenes_docker. Solo se cargan una única vez:

1. Abrir una consola (**PowerShell** o **cmd**) en la computadora.
2. Ejecutar estos comandos en el siguiente orden:

```
cd C:\proyecto_bd
docker load -i imagenes_docker\postgres_16.tar
docker load -i imagenes_docker\mysql_8.tar
docker load -i imagenes_docker\sqlserver_2022.tar
docker load -i imagenes_docker\oracle_23_slim.tar
```

IMPORTANTE: Cada comando puede llegar a tardar unos 10 a 20 segundos y mostrará un mensaje tipo: Loaded image: postgres:16-alpine*, como en la siguiente imagen*

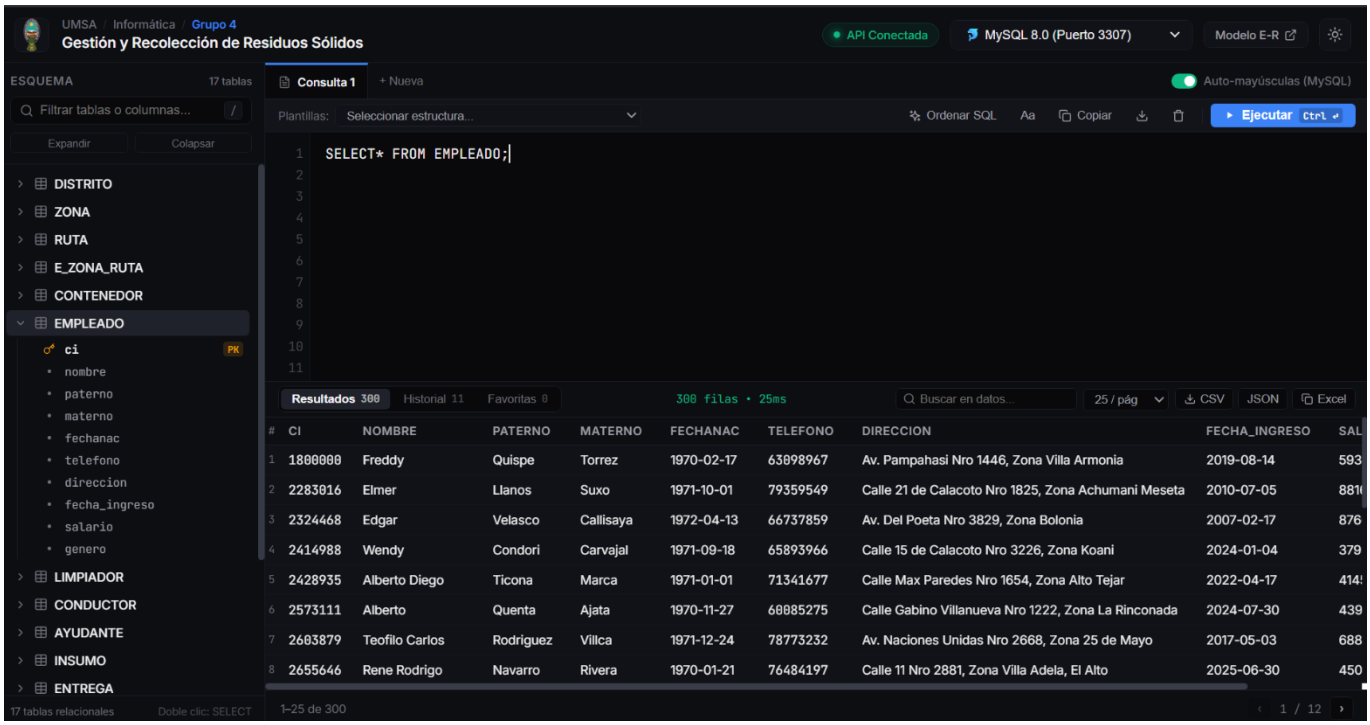


Paso 3: Encendido del Sistema con 1 Clic

1. En el Explorador de Archivos de Windows, entrar a C:\proyecto_bd\.
2. Hacer doble clic sobre el archivo ejecutable:

   iniciar_todo.bat

3. El script realizará automáticamente lo siguiente:
   - Encenderá los contenedores de base de datos en Docker.
   - Levantará el servidor Backend en el puerto 4000.
   - Levantará la interfaz web Frontend en el puerto 5173.
   - Abrirá automáticamente el navegador predeterminado en http://localhost:5173/.



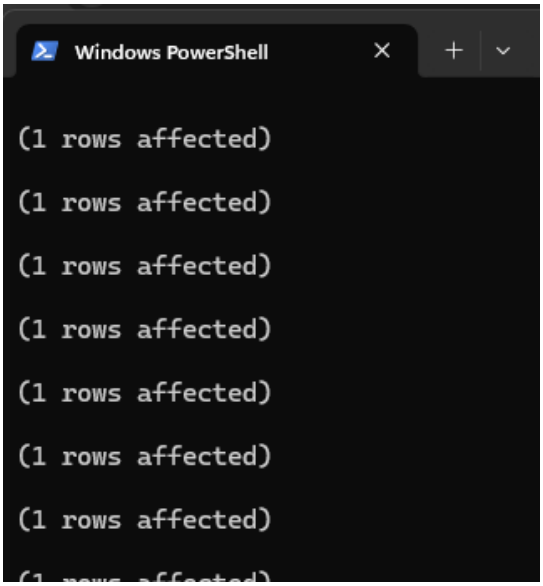
3. Inicialización Rápida de SQL Server y Oracle (Solo la primera vez)

Para que SQL Server registre su usuario unificado y sus 17 tablas:

A. Para SQL Server: Abre PowerShell en `C:\proyecto_bd` y ejecuta:

```
docker exec -i contenedor_sqlserver /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P "AdminPasswordGrupo4!" -C -i /docker-init/03_sqlserver_init.sql
```

(Si diera error de ruta en versiones anteriores de Docker, ejecutar con tools estándar: docker exec -i contenedor_sqlserver /opt/mssql-tools/bin/sqlcmd -S localhost -U sa -P "AdminPasswordGrupo4!" -i /docker-init/03_sqlserver_init.sql).



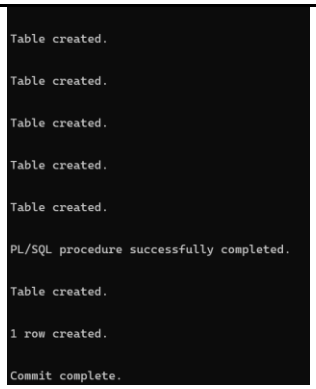
B. Para Oracle 23ai Free:

1. **Crear y autorizar al usuario:**

```
@"
ALTER SESSION SET CONTAINER = FREEPDB1;
CREATE USER grupo4_recoleccion IDENTIFIED BY grupo4;
GRANT CONNECT, RESOURCE, DBA TO grupo4_recoleccion;
ALTER USER grupo4_recoleccion QUOTA UNLIMITED ON USERS;
EXIT;
"@ | docker exec -i contenedor_oracle sqlplus -s sys/grupo4@FREEPDB1 as sysdba
```

2. **Cargar las tablas y los registros:**

```
docker exec -i contenedor_oracle sqlplus -s grupo4_recoleccion/grupo4@FREEPDB1 "@/container-entrypoint-initdb.d/01_init.sql"
```



4. Apagado Limpio

Cuando finalice la revisión o clase:

1. Hacer doble clic sobre el archivo apagar_todo.bat en C:\proyecto_bd\.
2. Esto liberará toda la memoria RAM, detendrá los motores de Docker y cerrará los procesos de Node.js limpiamente.

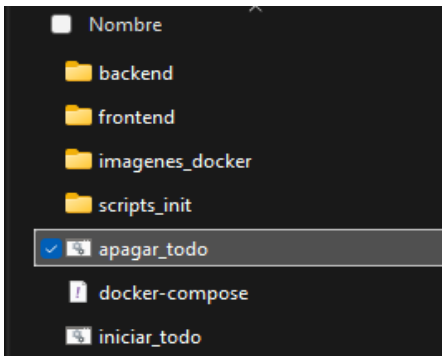


Tabla Resumen de Puertos y Credenciales para el Grupo

Motor	Entorno	Puerto Externo	Base de Datos / Servicio	Usuario	Contraseña
PostgreSQL	Docker	5433	bd_recoleccion	grupo4_recoleccion	grupo4
MySQL	Docker	3307	bd_recoleccion	grupo4_recoleccion	grupo4
SQL Server	Docker	1433	bd_recoleccion	grupo4_recoleccion	grupo4
Oracle 23ai Free	Docker	1522	FREEPDB1	grupo4_recoleccion	grupo4

PARTE II: APLICACIÓN PRÁCTICA (ANÁLISIS COMPARATIVO ORACLE VS. POSTGRESQL)

1. Análisis de Sintaxis y Capacidades

Matriz Comparativa de Sintaxis

Elemento Lógica	Oracle (PL/SQL)	PostgreSQL (PL/pgSQL)	Diferencias Clave
Delimitadores de Bloque	IS ... BEGIN ... END; /	AS $$ ... BEGIN ... END; $$ LANGUAGE plpgsql;	PostgreSQL encapsula el cuerpo de la función o procedimiento dentro de *Dollar-Quoting* ($$) y exige declarar explícitamente el lenguaje procedural (plpgsql).
Funciones	CREATE OR REPLACE FUNCTION nombre(p1 IN tipo) RETURN tipo IS ...	CREATE OR REPLACE FUNCTION nombre(p1 tipo) RETURNS tipo AS $$ ...	En PostgreSQL se usa la palabra clave RETURNS (con S), los parámetros no requieren obligatoriamente el indicador IN, y devuelven tipos escalares, tablas (RETURNS TABLE) o conjuntos (SETOF).
Procedimientos Almacenados	CREATE OR REPLACE PROCEDURE nombre(p1 IN tipo) IS ...	CREATE OR REPLACE PROCEDURE nombre(p1 tipo) LANGUAGE plpgsql AS $$ ...	PostgreSQL incorporó verdaderos procedimientos (CREATE PROCEDURE) con soporte nativo de transacciones (COMMIT/ROLLBACK internos) a partir de la versión 11. Ambos se ejecutan mediante CALL.
Triggers (Disparadores)	CREATE OR REPLACE TRIGGER nombre BEFORE INSERT ON tabla FOR EACH ROW BEGIN ... END;	Requiere dos pasos: 1. Crear función disparadora (RETURNS TRIGGER). 2. CREATE TRIGGER trg_nombre BEFORE INSERT ON tabla FOR EACH ROW EXECUTE FUNCTION funcion_trigger();	En Oracle la lógica de ejecución reside dentro de la misma declaración del trigger. En PostgreSQL es obligatorio desacoplar la lógica en una función RETURNS TRIGGER reutilizable.
Variables de Fila en Triggers	:NEW.columna y :OLD.columna	NEW.columna y OLD.columna	En Oracle las variables de tupla usan prefijo de dos puntos (:), mientras que en PostgreSQL se usan variables de registro sin dos puntos (NEW, OLD).
Cursores	CURSOR c_nom IS SELECT ...; OPEN c_nom; FETCH c_nom	c_nom CURSOR FOR SELECT ...; OPEN c_nom;	PostgreSQL permite cursores vinculados (bound) y no vinculados (REFCURSOR). Ambos admiten bucles

	INTO ...; CLOSE c_nom;	FETCH c_nom INTO ...; CLOSE c_nom;	automáticos de cursor: FOR fila IN c_nom LOOP ... END LOOP;.

Límites, Restricciones y Anidamiento

- Niveles de Anidamiento (*Nesting Levels*):
    - Oracle: Limita el anidamiento directo de triggers a un máximo de 32 niveles en cascada para prevenir desbordamientos de pila.
    - PostgreSQL: No impone un límite estricto de software en triggers en cascada; el anidamiento está acotado por el parámetro de memoria del servidor max_stack_depth (usualmente 2 MB, configurable). Si se superan los recursos de pila, PostgreSQL aborta la transacción arrojando stack depth limit exceeded.
- Capacidad de Almacenamiento por Registro:
    - Oracle: Límite tradicional por fila de 8 KB en bloques estándar (columnas VARCHAR2 limitadas a 4,000 bytes o 32,767 bytes si se habilita *Extended Data Types*). Tipos mayores requieren CLOB/BLOB.
    - PostgreSQL: Emplea la técnica TOAST (*The Oversized-Attribute Storage Technique*), permitiendo que campos largos comprimidos alcancen hasta 1 GB por celda de forma transparente, con un tamaño máximo teórico por tabla de 32 TB.

Manejo de Excepciones y Errores

- Oracle: Se apoya en bloques EXCEPTION WHEN nombre_excepcion THEN ... o WHEN OTHERS THEN. Los errores definidos por el usuario se disparan mediante RAISE_APPLICATION_ERROR(-20001, 'Mensaje') requiriendo códigos negativos entre -20000 y -20999.
- PostgreSQL: Dispone del estándar BEGIN ... EXCEPTION WHEN condicion THEN ... END;. Los errores se emiten mediante RAISE EXCEPTION 'Mensaje de error' o asociando códigos SQLSTATE específicos (RAISE EXCEPTION '...' USING ERRCODE = 'check_violation').

Ventajas y Desventajas

Gestor	Ventajas	Desventajas
PostgreSQL	Código abierto (sin costos de licenciamiento corporativo), extensibilidad completa (soporte nativo de JSON/JSONB, GIS con PostGIS, tipos personalizados), arquitectura moderna de procesos, cumplimiento estricto del estándar ANSI SQL.	Curva de afinamiento de memoria en alta concurrencia (shared_buffers, work_mem), mantenimiento periódico requerido para tablas con alto volumen de escrituras (VACUUM / *Autovacuum*).
Oracle Database	Motor empresarial consolidado, optimizador de consultas altamente sofisticado basado en costos (CBO), particionamiento de tablas y compresión avanzada, herramientas robustas de alta disponibilidad (*Real Application Clusters* / *Data Guard*).	Costos de licenciamiento prohibitivos para laboratorios pequeños, alto consumo de recursos de memoria RAM (la versión mínima Slim requiere 1.5 a 2 GB por sí sola), sintaxis procedural propia muy dependiente de la plataforma.

2. Investigación de Características Específicas

- Declaración y Ámbito de Variables:
    - En Oracle (PL/SQL), las variables se definen entre la cabecera y el cuerpo (IS / AS ... BEGIN), asignándose con el operador de asignación := o inicializándose por defecto con DEFAULT.
    - En PostgreSQL (PL/pgSQL), es indispensable contar con una sección explícita DECLARE previa al BEGIN. Admite asignaciones tanto con := como con =.
- Control de Transacciones dentro de Rutinas:
    - En Oracle, los procedimientos pueden contener sentencias COMMIT o ROLLBACK autónomas o globales.
    - En PostgreSQL, las funciones jamás pueden ejecutar transacciones autónomas (COMMIT o ROLLBACK están prohibidos dentro de una FUNCTION porque toda función corre dentro de la transacción externa que la llamó). El control transaccional explícito en PostgreSQL solo es legal dentro de CREATE PROCEDURE.
- Triggers de Nivel de Sentencia vs. Nivel de Fila:
    - Ambos gestores soportan triggers a nivel de sentencia (FOR EACH STATEMENT) y a nivel de fila (FOR EACH ROW). Sin embargo, en PostgreSQL, si la función del trigger devuelve NULL en un trigger BEFORE EACH ROW, la inserción o actualización de esa fila en particular se cancela silenciosamente sin disparar error.

3. Migración del Script de Código a la Otra Plataforma

El proceso metodológico seguido para migrar el modelo relacional desde Oracle a PostgreSQL abarcó 5 fases de conversión sintáctica:

1. Tipos de Datos Numéricos: Reemplazo sistemático de NUMBER(4) e identificadores enteros a INT, NUMBER(8) a INT, y montos monetarios NUMBER(8,2) a DECIMAL(8,2).
2. Cadenas de Caracteres: Sustitución de VARCHAR2(n) por el estándar ANSI VARCHAR(n).
3. Manejo Temporal: Conversión de funciones SYSDATE a CURRENT_DATE, CURRENT_TIMESTAMP a NOW(), y sentencias de inserción con TO_DATE('YYYY-MM-DD', 'YYYY-MM-DD') a literales de fecha nativos estándar ISO-8601 ('YYYY-MM-DD').
4. Tratamiento de Mayúsculas / Minúsculas: En Oracle los identificadores de tablas y columnas no delimitados se almacenan internamente en mayúsculas (EMPLEADO). En PostgreSQL, los identificadores se convierten automáticamente a minúsculas (empleado) a menos que se encierren entre comillas dobles.

5. Carga y Verificación: Se garantizó el orden de dependencia referencial: tablas maestras independientes (DISTRITO, INSUMO, CENTRO_DESTINO, EMPLEADO) se cargan antes que las tablas dependientes y las intermedias históricas (ABARCA, CONDUCE, ENTREGA).

4. Implementación en la Plataforma: Tres Ejercicios Resueltos de Cada Elemento (Proyecto Recolección de Residuos)

A continuación tienes los 3 ejercicios resueltos de cada requerimiento, con el código exacto en sintaxis PostgreSQL y su respectiva sentencia de prueba lista para copiar y ejecutar en tu panel web

==========

ÍNDICE DE CÓDIGO IMPLEMENTADO (POSTGRESQL - GRUPO 4)

---

1. FUNCIONES (PL/pgSQL)

1.1. fn_calcular_antiguedad_empleado

1.2. fn_total_capacidad_ruta

1.3. fn_verificar_vigencia_licencia

2. PROCEDIMIENTOS ALMACENADOS (PL/pgSQL)

2.1. sp_registrar_entrega_insumo

2.2. sp_cambiar_estado_vehiculo

2.3. sp_incremento_salarial_categoria

3. CURSORES ANIDADOS (PL/pgSQL)

3.1. Cursor Anidado: Distritos, Zonas y Rutas Asociadas

3.2. Cursor Anidado: Vehículos Operativos y Bitácora de Recolección

3.3. Cursor Anidado: Trazabilidad de Centros de Destino y Camiones

4. TRIGGERS (DISPARADORES)

4.1. trg_validar_salario_minimo (Tabla EMPLEADO)

4.2. trg_control_stock_insumos (Tabla ENTREGA)

4.3. trg_auditoria_cambio_salario (Tabla EMPLEADO y auditoria_salarios)

==========